



NoSQL Storage

Grundlagen Verteilter Systeme

Michael Schöttner

SS 2024

- ACID vs BASE
- NoSQL databases
- Google Bigtable
- HBase

- Data stored in columns and tables
- Relationships represented by data
- SQL = Data Manipulation and Definition Language
- Query optimization engine and indexes
- ACID transactions
- ...

DBMS = Database Management System

- Scalability → (distributed) transactions are not scalable
- ACID properties → limit concurrency
- No distributed processing frameworks built on top
- Fixed schema
- ...

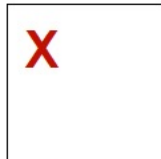
- ACID (Atomicity, Consistency, Isolation, Durability)
 - Strong consistency
 - Limited availability
 - Pessimistic approach
 - Complex locking schemes
- Distributed transactions require 2 Phase-Commit
 - Need to coordinated transaction parts on all involved servers → slow

- **BASE (Basically Available, Soft state & Eventual consistency)**
 - Focus is on availability and scalability
 - Weak consistency; typically eventual consistency
 - Simple and fast
 - Used in NoSQL databases

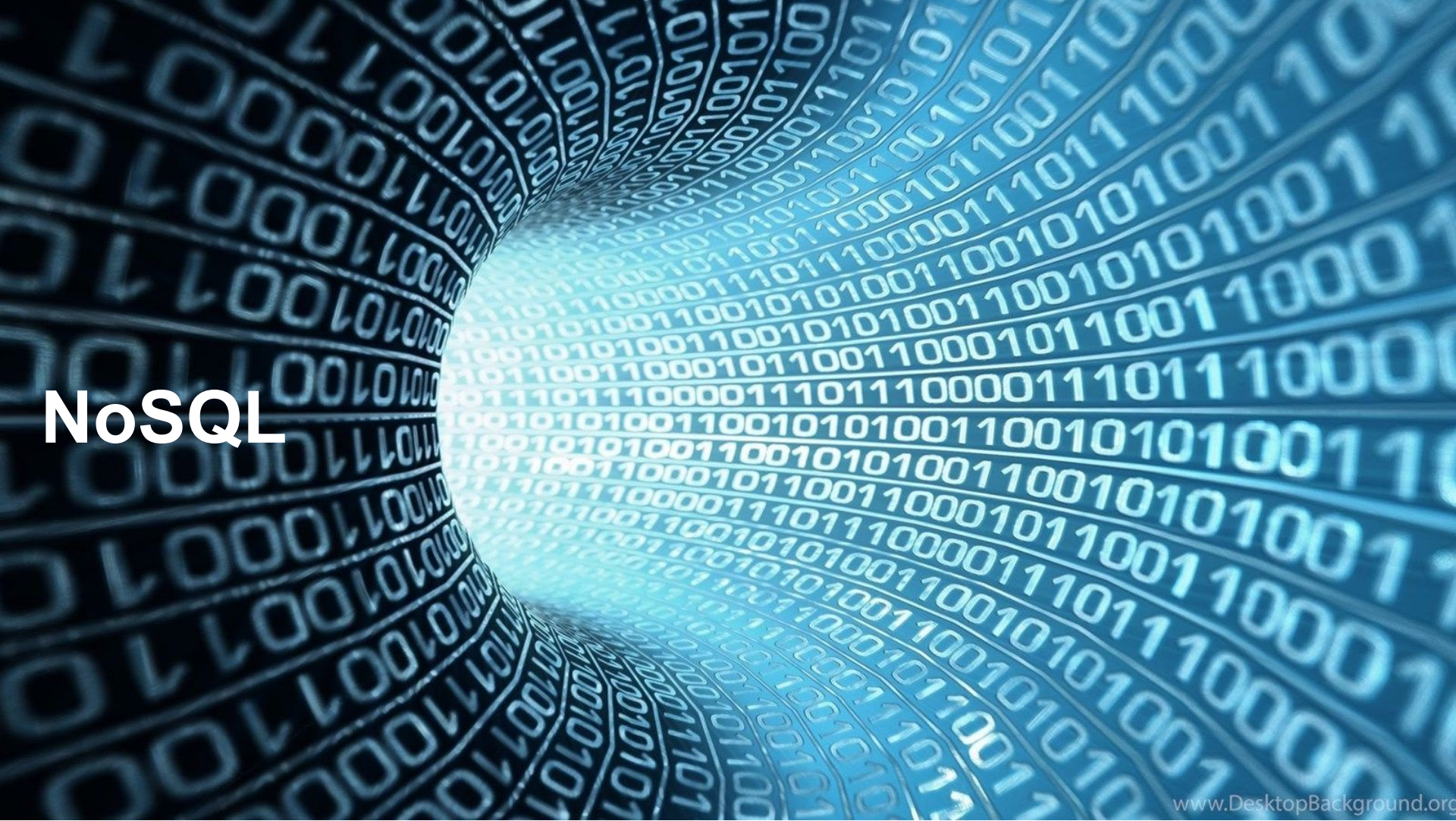
- **Literature**
 - „Cluster-based scalable network services“, Brewer et. al., Symposium on Operating Systems Principles (SOSP), France, 1997.
 - „BASE: An Acid Alternative“, Pritchett (Ebay), ACM Queue, Volume 6, Issue 3, May 2008.

- Ebay auction with many bids
- Must each bidder see each bid?
- Is it important for each bidder to see all bids in the same order?
- Obviously, each bidder needs to see the highest bid
 - But slightly inconsistent bidding histories are not a problem
 - Weaker consistency allows better scalability

- After uploading a Youtube videos it is not visible immediately for all people
- The update of a photo on a Facebook page is delayed and some users only see →



- Amazon has implemented the shopping card in the past using SQL → did not scale
 - As a result they developed Amazon Dynamo
 - A scalable and highly available key value store
 - To be discussed later

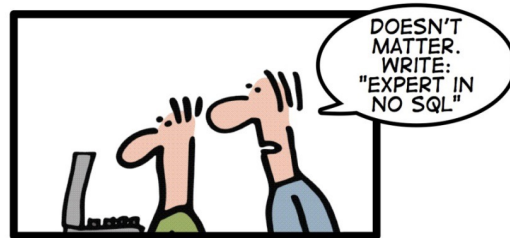
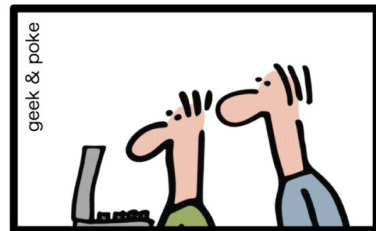


NoSQL

NoSQL

- NoSQL = Not Only SQL
- From www.nosql-database.org:
 - NoSQL definition: next generation database management systems mostly addressing some of the points: being non-relational, distributed, open-source and horizontally scalable.

HOW TO WRITE A CV



Leverage the NoSQL boom

- Large data volumes
- Scalable replication and distribution
 - Potentially thousands of machines
 - Potentially distributed around the world
- Queries need to return answers quickly
- Mostly queries, only few updates (asynchronous inserts & updates)
- BASE instead of ACID

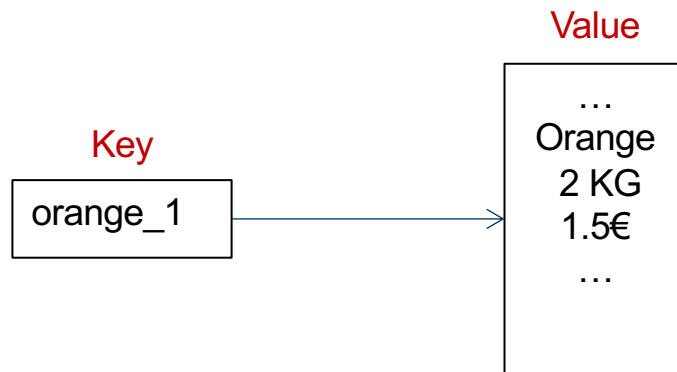
- Column Store – each storage block contains data from only one column (RDBMS use mainly row-based storage)
- Document Store – stores documents made up of tagged elements
- Key-Value Store – Hash table of keys
- Graph Databases – Multi-relational graph
- Vector Databases – stores data as high-dimensional vectors

- Inspired by Google Bigtable
- Bigtable stores (sparse) data
- Partitions data using keys
- Multi-dimensional tables
- Accessing whole columns (record type) is very fast
- Examples: Hbase, Bigtable, Cassandra

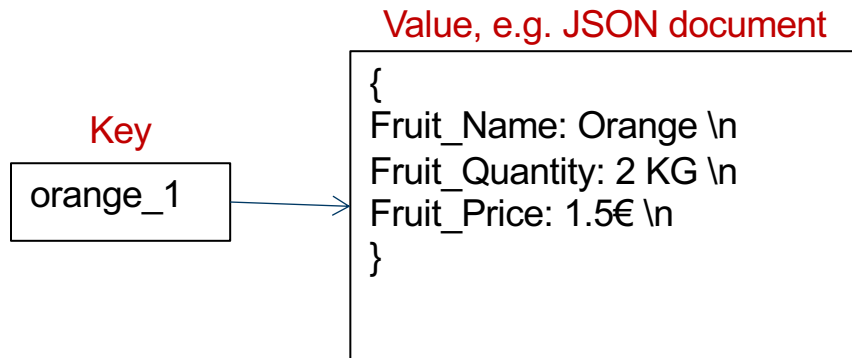
The diagram shows a table with three rows and three columns. Above the table, the text 'ROW ID' has an arrow pointing to the first column. The text 'COLUMNS' has three arrows pointing to the three columns. To the right of the table, the text 'Value' has an arrow pointing to the first row's third column cell.

apple	1.0 KG	2.2 €
banana	2 KG	2.5 €
orange	10KG	9 €

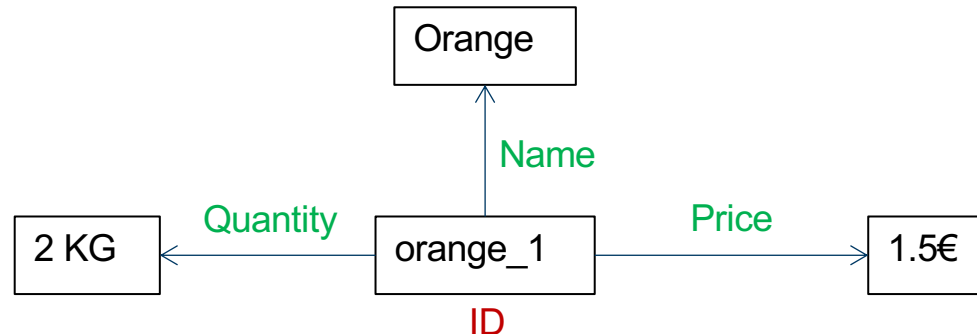
- Inspired by Amazon Dynamo
- Stores tuples (key, value)
 - Keys are user-defined strings, unique, and hashed
 - Value is typically a “blob” (object) – storage does not know the data type
- Very fast access for single keys
- Examples: Amazon Dynamo, Redis, DXRAM, ...



- Just like key-value stores, but value is understood by the database
- Data is structured based on a specific schema that describes the document
- More sophisticated ways to query the data, including contents in the document
- Examples: MongoDB, CouchDB, ...

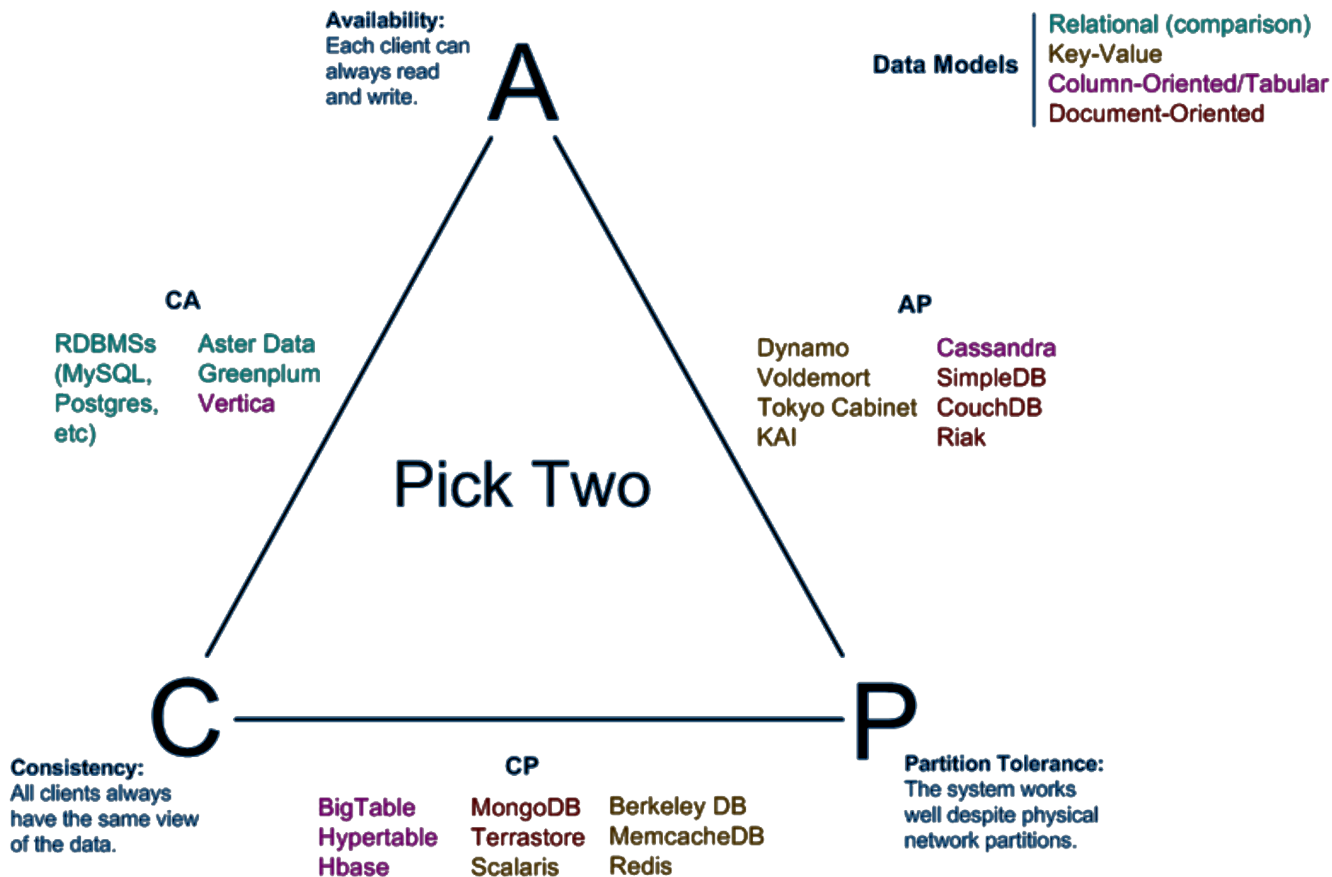


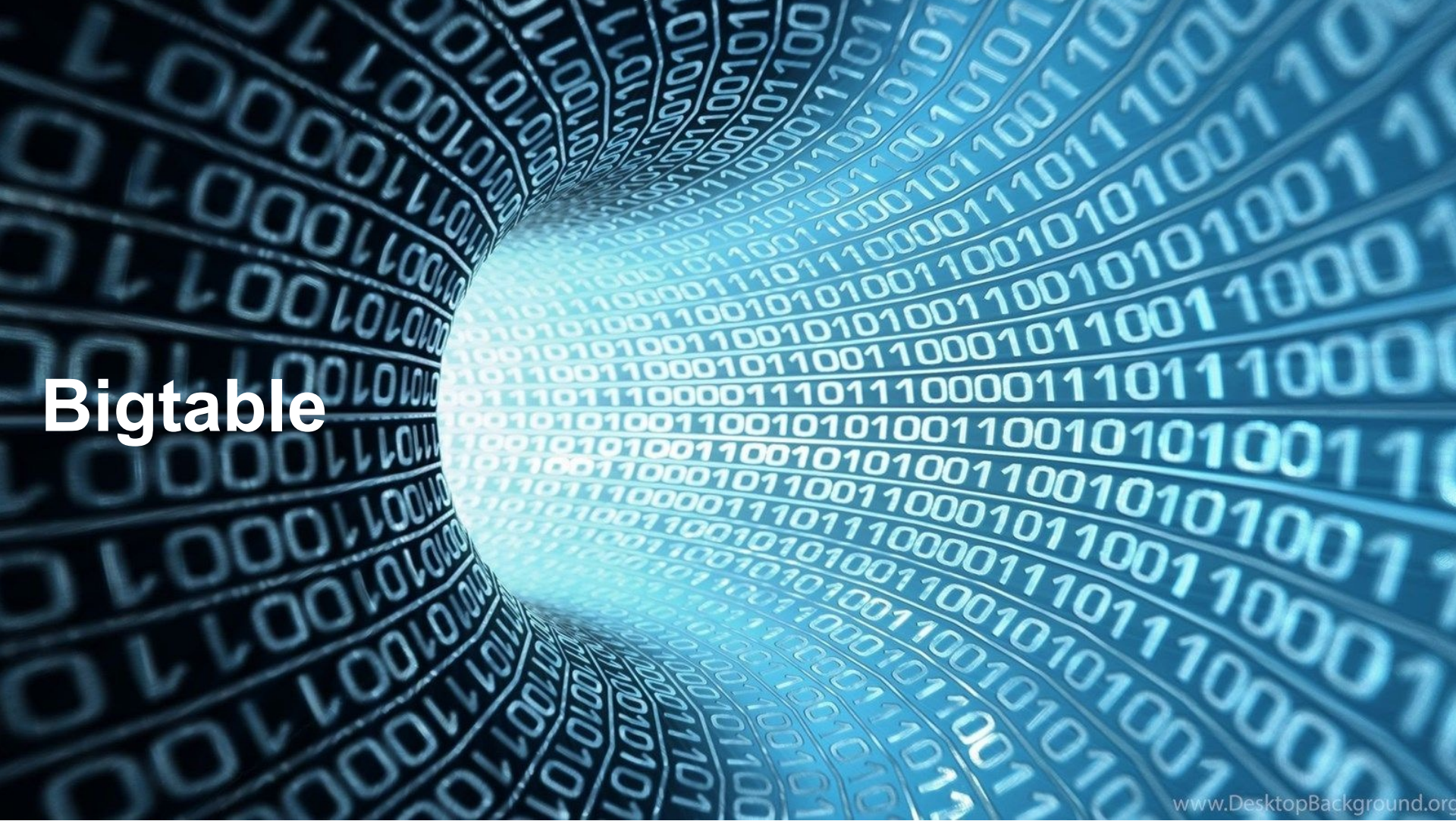
- Younger type of NoSQL database type
- Database is a multi-relational graph
- Used and inspired by social media graphs
- Relations are of primary importance in graph stores
- Examples: Giraph, GraphX (see later), ...



CAP (again)

Visual Guide to NoSQL Systems





Bigtable

- NoSQL database for google applications:

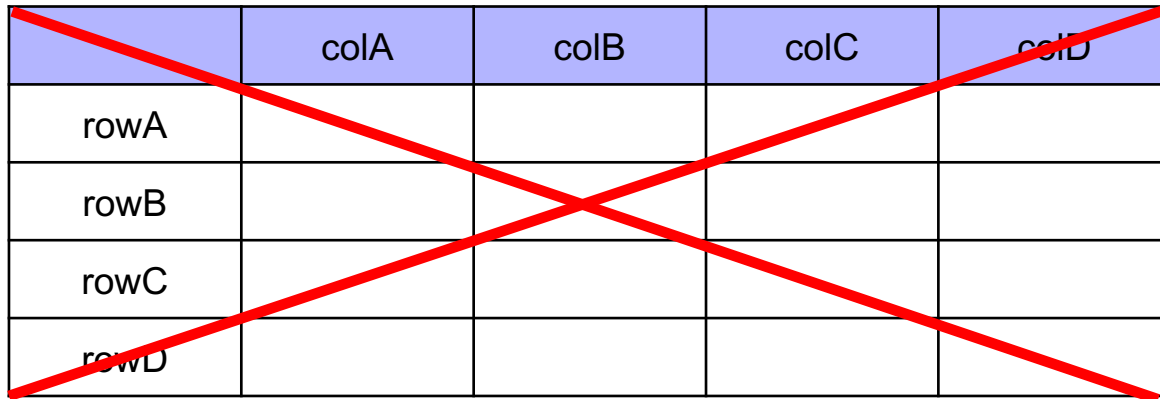
- Google Maps
- Google Books
- Google Earth
- Gmail
- ...

- F. Chang et. al., “Bigtable: A Distributed Storage System for Structured Data”, ACM Transactions on Computer Systems (TOCS), Vol. 26, No. 2, June 2006.



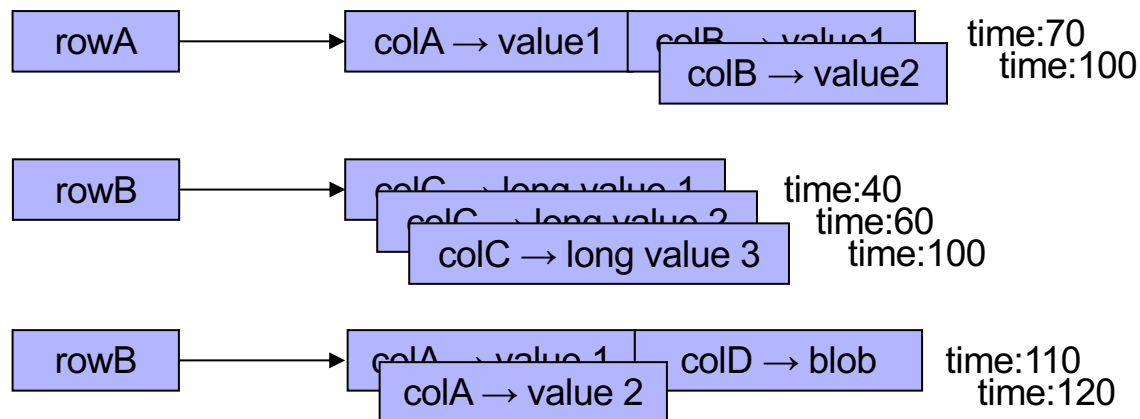
- Bigtable is also designed for fast appends
 - Rows are typically added but seldom updated
 - Columns are updated by adding a new column with a new timestamp
- Google Bigtable is built on top of GFS
 - Tables are stored in GFS
 - GFS is responsible for persistence (replication & load balancing)
- One Bigtable cluster (= cell) is used for one or several applications
 - 10 – 1.000 servers per cell
 - Google has several hundreds of cells

- No traditional database tables
- Rows do not have a fixed size and also not a fixed number of attributes



	colA	colB	colC	colD
rowA				
rowB				
rowC				
rowD				

- Bigtable is implementing a multi-dimensional sparse map
- Each row stores only needed attributes/columns
- Timestamps are used for modified attributes



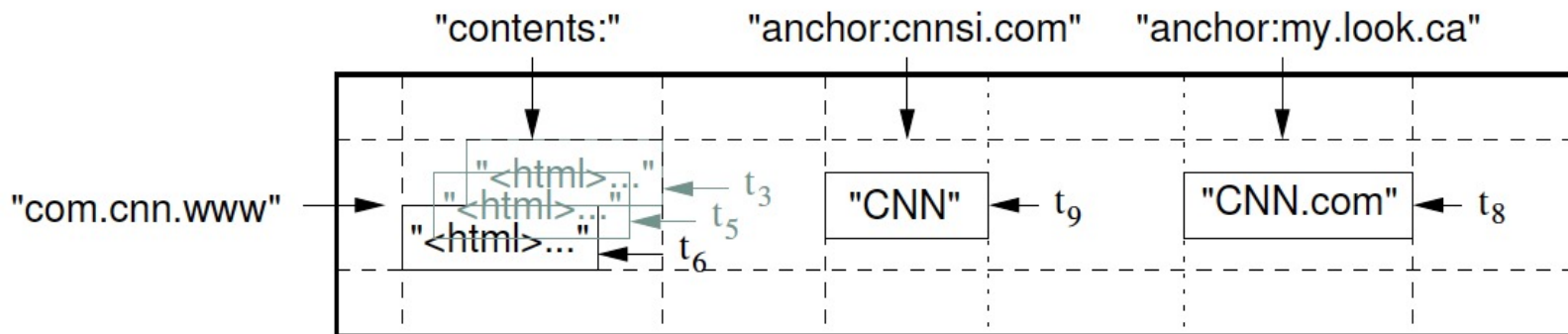
- Rows have a unique name
 - e.g. www.uni-duesseldorf.de
- **Each row access is atomic**
 - You always read/write full rows
- Rows are sorted in lexicographical order within one table
 - Motivation: if a table is partitioned, lexicographically similar rows are stored in the same partition or at least in a nearby partition
 - www.uni-duesseldorf.de is near to <http://www.uni-duesseldorf.de/home/studium-und-lehre-an-der-hhu.html>
- Rows are not divided during partitioning

- Column names have two levels
 - Family name and qualifier name
 - Syntax: `<family:qualifier>`
- Families must be defined during schema creation
 - Columns within one family have typically the same type
 - Data of one row within one family are often stored and compressed together
- Individual columns within one family can be created dynamically
- Objective: max. 100 column families

- 64-Bit integer
- Updates create new versions
 - Typically using system time as timestamps
 - Can also be set by application
- Configuration to limit number of versions defined during column family creation, e.g.
 - Store not more than N versions
 - Store only version that are not older than X seconds
- Many requests include timestamps

Example

- The row name is the inverted URL
- Column family “contents” stores the content of the web page
- Column family “anchor” stores text linking to this web page
 - The example below shows the row for the home page of CNN
 - It is referenced from Sports Illustrated <http://www.cnnsi.com> & MY-look <http://www.my.look.ca> → two columns

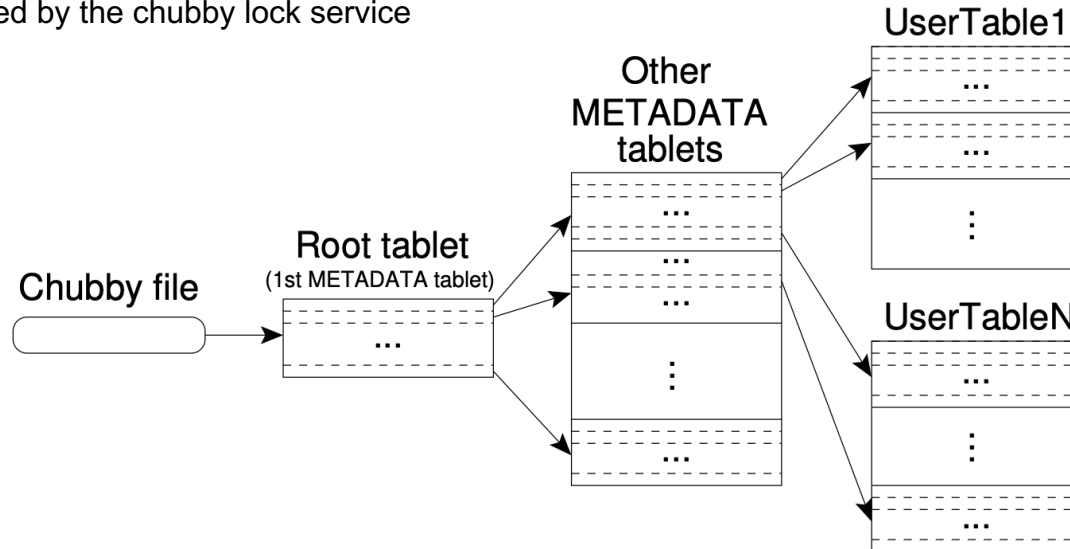


- Tables are partitioned into several tablets
- Tablet is the basic unit for partitioning and load balancing
- A tablet is a set of consecutive rows in a table
- Rows are never divided
- The typical tablet size is 1 GB

- Bigtable offers a rather basic API, SQL queries are not supported
- Function overview:
 - Create and delete tables and columns
 - Change metadata, e.g. access rights
 - Write and deleted values → single row transactions
 - Read values in given rows
 - Iterating over a set of rows
 - Using regular expression for accessing row and column names

- A Bigtable cluster/cell runs one master and several tablet servers
- Master: controls tablet servers
 - Manages schema
 - Assigns and migrates tablets
 - Controls garbage collection and load balancing
 - Clients are contacting the master not very often
- Tablet Server: storage tablets data
 - Clients directly contact tablet server for accessing data

- Clients need to lookup tablet servers
 - Tablet server locations are stored in Bigtable itself
 - A chubby file (chubby lock service) points to root tablet server
 - A chubby file is stored by the chubby lock service



- Each tablet is stored in one GFS file
- The tablet server typically stores the GFS primary replica
- This results in read and write operations to be executed on the local disk
- Tablet migration also implies migrating affected GFS chunks

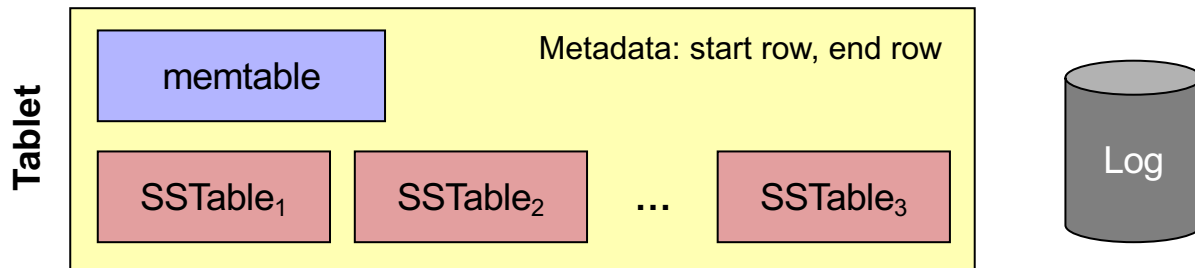
- Master controls tablet servers using chubby lock service
- Joining of a new tablet server
 - Creates an ID file (used as an ID for this server) in a given directory for the cell
 - Afterwards the new server requests a lock for the ID file
 - The lock is renewed periodically by the tablet server (the lock is lease based)
 - The master monitors the cell directory and can detect new ID files (new tablet servers)
- Leaving/crashing tablet server
 - Detected by master if an ID file is no longer locked
 - The master assigns the tablets to other tablet servers and the ID file is deleted

Implementation: Master failure

- We need a standby master which tries to get the lock on the ID file of the master stored in chubby lock
- If it gets the lock it takes over control
- The lock ensures that only one master is in control
- A new master can load metadata for tablets from Bigtable

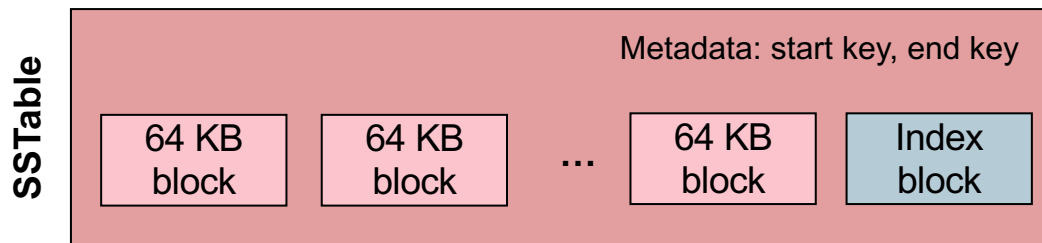
Implementation: Storing tablets

- **SSTable**: tablet data is stored in several SSTables (read-only) which are stored in GFS (one GFS file per SSTable)
- **memtable**: buffers sorted data in memory
→ allows asynchronous and aggregated writes (improves throughput)
- **Log**: persistent append-only log for all write accesses
 - Allows recovery of the memtable, in case of failures
 - The log is shared for all tables of a tablet server and is stored in GFS

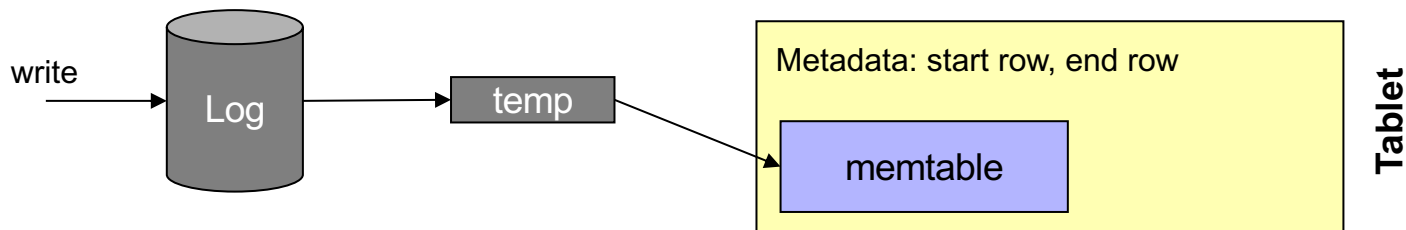


Implementation: Storing tablets

- SSTables store sorted key-value tuple sets (size 100 MB – 1 GB)
 - Key: triple <row, column, timestamp>
 - Value: data
- SSTables are read-only → no consistency issues
- SSTables are stored in 64kb blocks
- Index block is also sorted → allows binary search

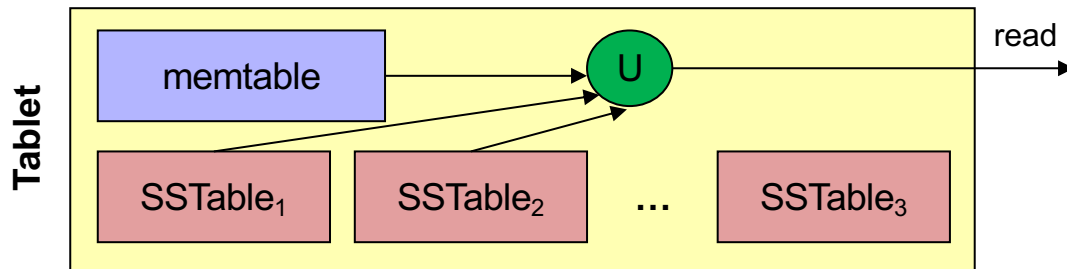


- Write access to a row is atomic
- Steps for an incoming write request
 - Tablet server checks access rights using chubby
 - Write a log record
 - If OK, saves data in memtable
 - Updated row is first stored in temp
 - If the row has been fully received it is copied to memtable



- Memtable is growing with each write
- The table is frozen if a threshold is reached, and a new table is created
- The frozen Memtable is written to disk as a new SSTable
 - This is also called “minor compaction”
 - This step is also recorded in the log (similar to a checkpoint)
 - Allows fast recovery → no need to analyze full log for recovering Memtable

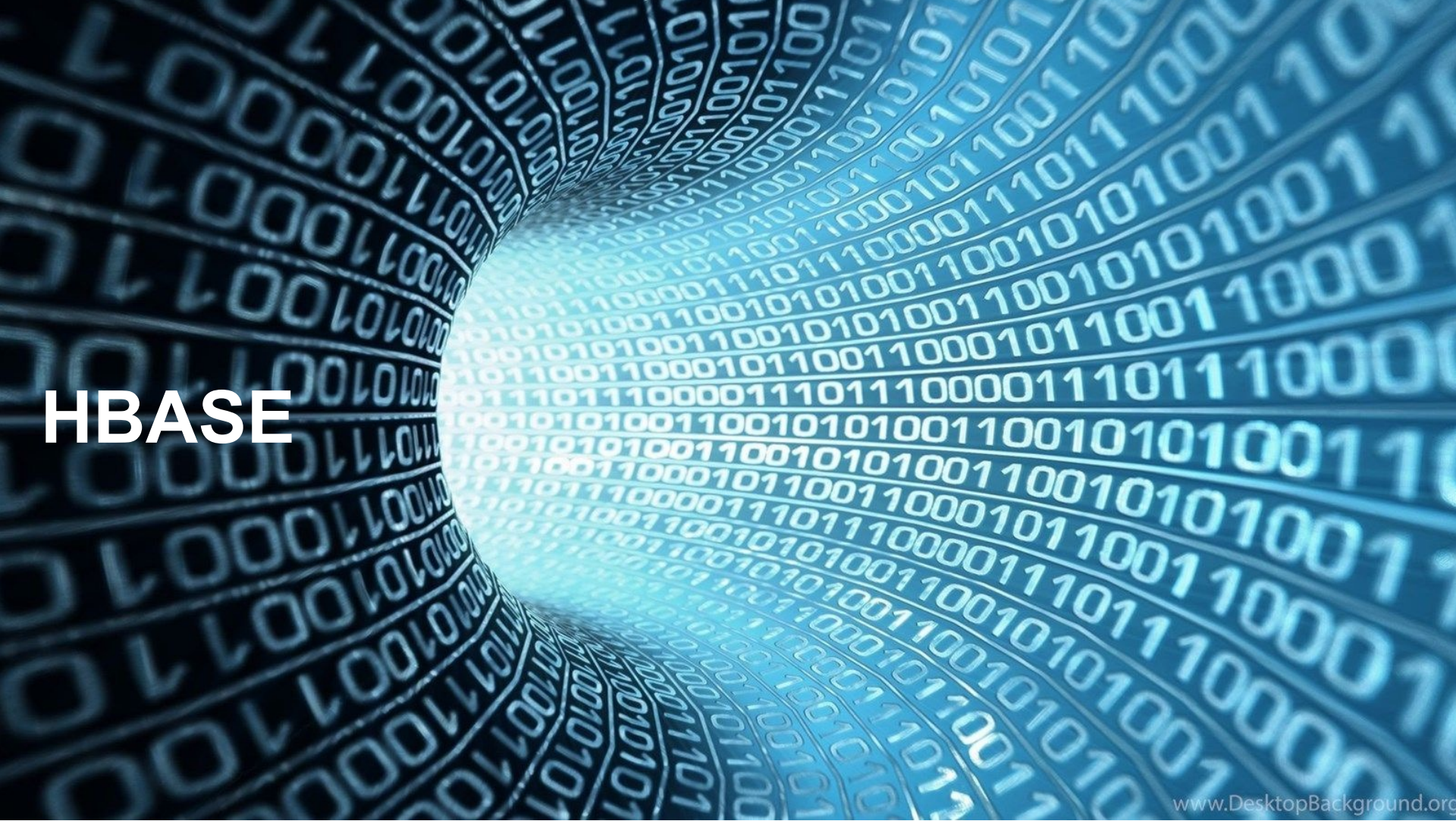
- Steps for an incoming read / read-range request
 - First access rights are checked using chubby
 - Tablet server uses index block of SSTables and memtable to find all relevant blocks
 - All needed blocks are bundled into a sorted view U
 - This is done using merge sort (blocks are already sorted)
 - Finally, a binary search can be applied to the sorted view



- Tuples to be deleted are marked
- Removing happens during the “major compaction”
 - Several overlapping SSTables and the memtable are compacted into a set of non-overlapping SSTables
 - Marked tuples are deleted during this process
 - This results in faster read accesses
 - The compaction runs concurrently, so read and write accesses are served in parallel

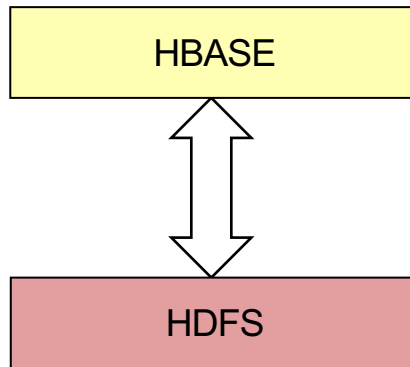
Implementation: table server failure

- No problem for SSTables → are stored and replicated in GFS
- But memtable is lost
 - Can be recovered from log file
 - Log file is also stored in GFS and thus also replicated

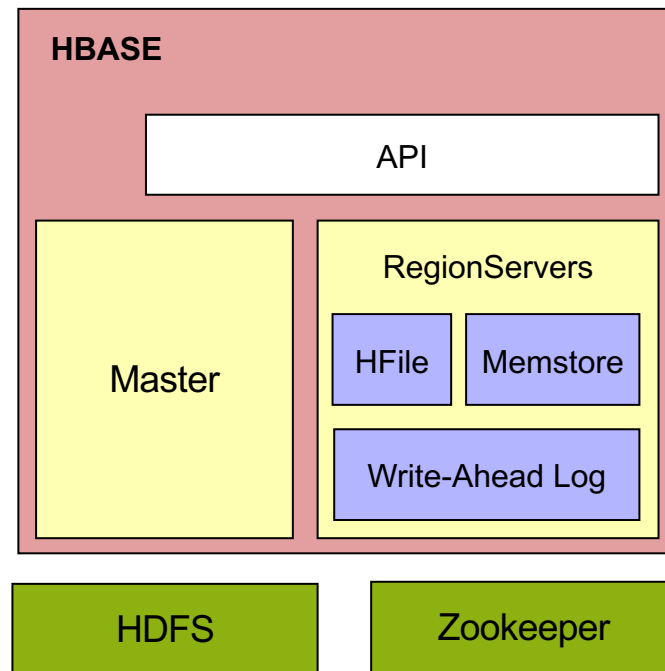


HBASE

- Open-source implementation of Bigtable
 - Written in Java
 - Data is also stored as a sparse, distributed, persistent multidimensional sorted map
- SQL interface (indirect via Spark, Hive, ...)
- HDFS is used as backend to store data
 - HDFS is very similar to GFS
 - Utilize replication and place servers close to data



- **Master**
 - Metadata management
 - Monitoring & load balancing
- **RegionServer process runs on storage nodes**
 - Manage key ranges (= regions) on a per table bases
 - Buffer I/O to multiple files on HDFS
 - Performs potential computations (and data filtering)
- **Zookeeper used for configuration & synchronization**



■ Memstore

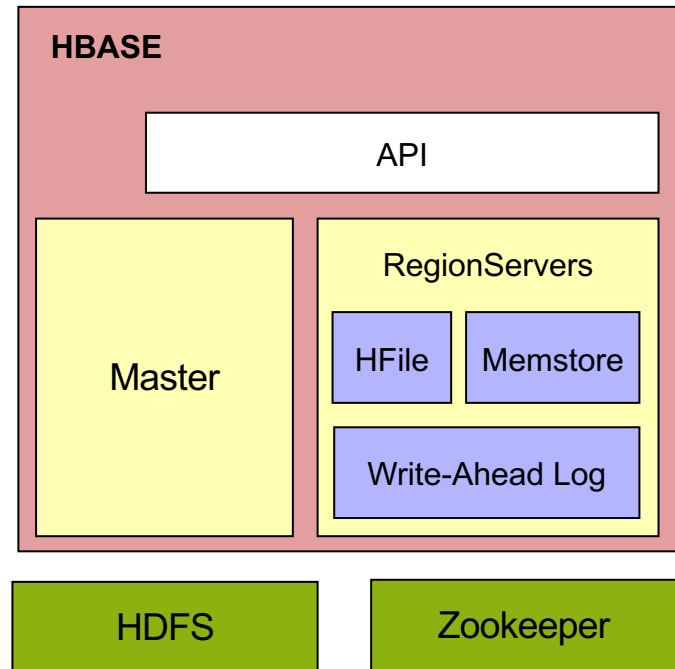
- One for each region
- In-memory write cache
- If filled, data is flushed to new HFiles

■ HFile (also called store files)

- Stores cells sorted by row key
- Append only; a deletion writes key type with tombstone marker
- Compaction process merges multiple store files

■ Write-Ahead Log

- Record all planned data changes before doing them
- Ensure durability by enabling replay when server crashes



- NoSQL storage is designed for large data volumes
- Relaxed consistency → BASE instead of ACID
- Scalable replication and distribution
- Google Bigtable and Apache HBASE have very similar architecture and features